

$\frac{1}{2}$

ToDoこな

メモ

- クラス内の変数は、すべて、「get_変数名」という名前のメソッドを介してアクセスするように設計してある
- すべてのクラスに関する関数は、クラスの外で定義してある
`os.chdir(os.path.dirname(os.path.abspath(file)))`

 c0a25046bf / ProjExD_Group14

 | 

 **Code**  Issues  Pull requests **1**  Agents  Actions  Projects  Wiki  Security and quality  Insights

 On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#). 

  C0A25046/ステ...  **ProjExD_Group14** / koukaton_up.py 

 Go to file  T 

 **c0a25046bf** ギミックを1層1つに変更 7e23673 · 47 minutes ago 

331 lines (258 loc) · 11.5 KB

Code **Blame**

 **Raw**   

```
1  import pygame
2  import sys
3  import os
4  import random
5
6  # カレントディレクトリの固定
7  os.chdir(os.path.dirname(os.path.abspath(__file__)))
8
9  WIDTH, HEIGHT = 600, 800
10 FPS = 60
11 GRAVITY = 0.6
12
13 BLACK = (20, 20, 25)
14 BLUE = (100, 150, 255)
15 WHITE = (255, 255, 255)
16 GOLD = (255, 215, 0)
17
18 # ギミック床の色定義を復活
19 COLOR_NORMAL = (120, 120, 120)    # 緑 (通常)
20 COLOR_FAKE = (180, 0, 180)        # 紫 (すり抜ける)
21 COLOR_ICE = (100, 200, 255)       # 水色 (滑る)
22 COLOR_TRAMPOLINE = (255, 140, 0)  # オレンジ (大ジャンプ)
23 COLOR_TRAP = (255, 50, 50)        # 赤 (最初に戻る)
24 COLOR_RED_UI = (255, 100, 100)    # UI用赤
25
26 # 種類から色を取得する辞書
27 COLOR_MAP = {
28     "normal": COLOR_NORMAL,
29     "fake": COLOR_FAKE,
30     "ice": COLOR_ICE,
31     "trampoline": COLOR_TRAMPOLINE,
32     "trap": COLOR_TRAP
33 }
34
35 # =====
36 # 1. データ構造 (クラス) の定義
37 # =====
38
39 class Player:
40     def __init__(self):
41         self._image = pygame.Surface((30, 40))
42         self._image.fill(BLUE)
43         self._rect = self._image.get_rect(center=(WIDTH // 2, HEIGHT - 100))
44
45         self._vel_x = 0
46         self._vel_y = 0
47         self._on_ground = False
48
```

```
49     self._is_charging = False
50     self._charge_power = 0
51     self._max_charge = 20.0
52     self._direction = 1
53
54     self._is_clear = False
55     self._on_ice = False
56     self._is_reset = False
57
58     # --- ゲッター ---
59     def get_image(self): return self._image
60     def get_rect(self): return self._rect
61     def get_vel_x(self): return self._vel_x
62     def get_vel_y(self): return self._vel_y
63     def get_on_ground(self): return self._on_ground
64     def get_is_charging(self): return self._is_charging
65     def get_charge_power(self): return self._charge_power
66     def get_max_charge(self): return self._max_charge
67     def get_direction(self): return self._direction
68     def get_is_clear(self): return self._is_clear
69     def get_on_ice(self): return self._on_ice
70     def get_is_reset(self): return self._is_reset
71
72     # --- セッター ---
73     def set_vel_x(self, value): self._vel_x = value
74     def set_vel_y(self, value): self._vel_y = value
75     def set_on_ground(self, value): self._on_ground = value
76     def set_is_charging(self, value): self._is_charging = value
77     def set_charge_power(self, value): self._charge_power = value
78     def set_direction(self, value): self._direction = value
79     def set_is_clear(self, value): self._is_clear = value
80     def set_on_ice(self, value): self._on_ice = value
81     def set_is_reset(self, value): self._is_reset = value
82
83
84     class Platform:
85     def __init__(self, x, y, w, h, color, plat_type="normal"):
86         self._image = pygame.Surface((w, h))
87         self._image.fill(color)
88         self._rect = self._image.get_rect(topleft=(x, y))
89         self._type = plat_type
90
91         def get_image(self): return self._image
92         def get_rect(self): return self._rect
93         def get_type(self): return self._type
94
95
96     # =====
97     # 2. クラスを操作する関数
98     # =====
99
100     def update_player(player, keys, platforms, goal_block):
101         if not player.get_is_clear():
102             if player.get_on_ground():
103
104                 if player.get_on_ice():
105                     player.set_vel_x(player.get_vel_x() * 0.96)
106                 else:
107                     player.set_vel_x(0)
108
109                 if keys[pygame.K_LEFT] or keys[pygame.K_a]:
110                     player.set_direction(-1)
111                 if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
112                     player.set_direction(1)
113
114                 if keys[pygame.K_SPACE]:
115                     player.set_is_charging(True)
```

```
116         current_power = player.get_charge_power()
117         if current_power < player.get_max_charge():
118             player.set_charge_power(current_power + 0.4)
119     else:
120         if player.get_is_charging():
121             power = player.get_charge_power()
122             direction = player.get_direction()
123
124             player.set_vel_y(-power * 0.9 - 5)
125             player.set_vel_x(direction * (power * 0.4 + 2))
126
127             player.set_is_charging(False)
128             player.set_charge_power(0)
129             player.set_on_ground(False)
130             player.set_on_ice(False)
131
132     if not player.get_on_ground():
133         player.set_vel_y(player.get_vel_y() + GRAVITY)
134
135     rect = player.get_rect()
136
137     rect.x += player.get_vel_x()
138     if rect.left < 0:
139         rect.left = 0
140         player.set_vel_x(player.get_vel_x() * -0.5)
141         player.set_direction(1)
142     if rect.right > WIDTH:
143         rect.right = WIDTH
144         player.set_vel_x(player.get_vel_x() * -0.5)
145         player.set_direction(-1)
146
147     rect.y += int(player.get_vel_y())
148
149     goal_rect = goal_block.get_rect()
150     if rect.collidirect(goal_rect):
151         if player.get_vel_y() < 0:
152             rect.top = goal_rect.bottom
153             player.set_vel_y(0)
154             player.set_is_clear(True)
155
156     player.set_on_ground(False)
157     player.set_on_ice(False)
158
159     if player.get_vel_y() > 0:
160         for plat in platforms:
161             if plat.get_type() == "fake":
162                 continue
163
164             plat_rect = plat.get_rect()
165             if rect.collidirect(plat_rect):
166                 if rect.bottom <= plat_rect.top + player.get_vel_y() + 1:
167
168                     if plat.get_type() == "trampoline":
169                         rect.bottom = plat_rect.top
170                         player.set_vel_y(-25)
171                         player.set_on_ground(False)
172                         break
173
174                     if plat.get_type() == "trap":
175                         rect.center = (WIDTH // 2, HEIGHT - 100)
176                         player.set_vel_y(0)
177                         player.set_vel_x(0)
178                         player.set_is_reset(True)
179                         break
180
181                 rect.bottom = plat_rect.top
182                 player.set_vel_y(0)
```

```

183         player.set_on_ground(True)
184
185         if plat.get_type() == "ice":
186             player.set_on_ice(True)
187         else:
188             player.set_vel_x(0)
189         break
190
191
192 # =====
193 # 3. メインループ
194 # =====
195
196 ✓ def main():
197     pygame.init()
198     screen = pygame.display.set_mode((WIDTH, HEIGHT))
199     pygame.display.set_caption("Jump King - Colored Gimmicks")
200     clock = pygame.time.Clock()
201     font = pygame.font.SysFont(None, 36)
202     large_font = pygame.font.SysFont(None, 72)
203
204     player = Player()
205     platforms = []
206
207     platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, COLOR_NORMAL, "normal"))
208
209     TOTAL_FLOORS = 10
210     goal_y = (HEIGHT - 40) - (HEIGHT * TOTAL_FLOORS)
211     goal_block = Platform(0, goal_y, WIDTH, 40, GOLD, "goal")
212
213     for f in range(TOTAL_FLOORS):
214         floor_bottom = HEIGHT - (f * HEIGHT)
215         floor_top = HEIGHT - ((f + 1) * HEIGHT)
216
217         floor_plats = []
218         plat_y = floor_bottom - 150
219
220         while plat_y > floor_top + 50:
221             w = random.randint(60, 130)
222             x = random.randint(0, WIDTH - w)
223             floor_plats.append(Platform(x, plat_y, w, 20, COLOR_NORMAL, "normal"))
224             gap_y = random.randint(80, 130)
225             plat_y -= gap_y
226
227         if len(floor_plats) >= 2:
228             special_type = random.choice(["fake", "ice", "trampoline"])
229
230             # 10階の場合：ランダム特殊床1つ + トラップ床1つ
231             if f == TOTAL_FLOORS - 1:
232                 specials = random.sample(range(len(floor_plats)), 2)
233
234                 p_spec = floor_plats[specials[0]]
235                 # 種類に応じた色 (COLOR_MAP[special_type]) を適用
236                 floor_plats[specials[0]] = Platform(p_spec.get_rect().x, p_spec.get_rect().y, p_spec.get_rect().width, 20, COLOR_MAP[special_type])
237
238                 p_trap = floor_plats[specials[1]]
239                 # 赤色を適用
240                 floor_plats[specials[1]] = Platform(p_trap.get_rect().x, p_trap.get_rect().y, p_trap.get_rect().width, 20, COLOR_MAP["trampoline"])
241
242             # 1〜9階の場合：ランダム特殊床1つ
243             else:
244                 special_idx = random.choice(range(len(floor_plats)))
245                 p_spec = floor_plats[special_idx]
246                 floor_plats[special_idx] = Platform(p_spec.get_rect().x, p_spec.get_rect().y, p_spec.get_rect().width, 20, COLOR_MAP[special_type])
247
248         platforms.extend(floor_plats)
249

```

```
250 camera_y = 0
251 max_height = 0
252
253 running = True
254 while running:
255     for event in pygame.event.get():
256         if event.type == pygame.QUIT:
257             running = False
258
259     keys = pygame.key.get_pressed()
260
261     update_player(player, keys, platforms, goal_block)
262
263     p_rect = player.get_rect()
264
265     if player.get_is_reset():
266         camera_y = 0
267         player.set_is_reset(False)
268
269     SCROLL_TOP_MARGIN = 200
270     if p_rect.y - camera_y < SCROLL_TOP_MARGIN:
271         camera_y = p_rect.y - SCROLL_TOP_MARGIN
272
273     SCROLL_BOTTOM_MARGIN = 150
274     camera_easing_down = 0.08
275     if p_rect.bottom - camera_y > HEIGHT - SCROLL_BOTTOM_MARGIN:
276         target_camera_y = p_rect.bottom - (HEIGHT - SCROLL_BOTTOM_MARGIN)
277         camera_y += (target_camera_y - camera_y) * camera_easing_down
278
279     if camera_y > 0:
280         camera_y = 0
281
282     current_height = (HEIGHT - 40) - p_rect.bottom
283     if current_height > max_height:
284         max_height = current_height
285
286     current_floor = min((current_height // HEIGHT) + 1, TOTAL_FLOORS)
287     display_floor = max(1, current_floor)
288
289     screen.fill(BLACK)
290
291     goal_rect = goal_block.get_rect()
292     goal_draw_y = goal_rect.y - camera_y
293     if -100 < goal_draw_y < HEIGHT + 100:
294         screen.blit(goal_block.get_image(), (goal_rect.x, goal_draw_y))
295
296     for plat in platforms:
297         plat_rect = plat.get_rect()
298         draw_y = plat_rect.y - camera_y
299         if -50 < draw_y < HEIGHT + 50:
300             screen.blit(plat.get_image(), (plat_rect.x, draw_y))
301
302     player_draw_y = p_rect.y - camera_y
303     screen.blit(player.get_image(), (p_rect.x, player_draw_y))
304
305     if player.get_is_charging() and not player.get_is_clear():
306         gauge_width = (player.get_charge_power() / player.get_max_charge()) * 40
307         pygame.draw.rect(screen, COLOR_RED_UI, (p_rect.centerx - 20, player_draw_y - 15, gauge_width, 8))
308
309     arrow_x = p_rect.right + 5 if player.get_direction() == 1 else p_rect.left - 10
310     pygame.draw.rect(screen, WHITE, (arrow_x, player_draw_y + 15, 5, 5))
311
312     ui_text = font.render(f"Floor: {display_floor} / {TOTAL_FLOORS}    Max: {max_height // 10}m", True, WHITE)
313     screen.blit(ui_text, (10, 10))
314
315     if player.get_is_clear():
316         clear_text = large_font.render("CLEAR!!", True, GOLD)
```

```
317     text_rect = clear_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 50))
318     screen.blit(clear_text, text_rect)
319
320     sub_text = font.render("CONGRATULATIONS!", True, WHITE)
321     sub_rect = sub_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 + 10))
322     screen.blit(sub_text, sub_rect)
323
324     pygame.display.flip()
325     clock.tick(FPS)
326
327     pygame.quit()
328     sys.exit()
329
330 if __name__ == "__main__":
331     main()
```

c0a25046bf / ProjExD_Group14

🔍

👤

+

🕒

🔗

📄

📧

<> Code

🕒 Issues

🔗 Pull requests 1

🚗 Agents

🎬 Actions

📁 Projects

📖 Wiki

🛡 Security and quality

📈 Insights

🔔

 On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. [Review this update](#) and manage your preferences in your [GitHub account settings](#).

✕

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

🔗

base: main

⏪

compare: C0A25046/ステージギミック

✓ Able to merge. These branches can be automatically merged.

🔗 C0A25046/ステージギミック #1

No description available

🔗 View pull request

🕒 2 commits

📄 1 file changed

👤 1 contributor

Commits on May 19, 2026

ギミック追加

👤 c0a25046bf committed 1 hour ago

📄 2553f8d

<>

ギミックを1層1つに変更

👤 c0a25046bf committed 35 minutes ago

📄 7e23673

<>

Showing 1 changed file with 106 additions and 48 deletions.

Split Unified

154 koukaton_up.py		
12	12	
13	13	BLACK = (20, 20, 25)
14	14	BLUE = (100, 150, 255)
15		- GREEN = (120, 120, 120)
16		- RED = (255, 100, 100)
17	15	WHITE = (255, 255, 255)
18		- GOLD = (255, 215, 0) # ゴールブロック用の金色
	16	+ GOLD = (255, 215, 0)
	17	+
	18	+ # ギミック床の色定義を復活
	19	+ COLOR_NORMAL = (120, 120, 120) # 緑 (通常)
	20	+ COLOR_FAKE = (180, 0, 180) # 紫 (すり抜ける)
	21	+ COLOR_ICE = (100, 200, 255) # 水色 (滑る)
	22	+ COLOR_TRAMPOLINE = (255, 140, 0) # オレンジ (大ジャンプ)
	23	+ COLOR_TRAP = (255, 50, 50) # 赤 (最初に戻る)
	24	+ COLOR_RED_UI = (255, 100, 100) # UI用赤
	25	+
	26	+ # 種類から色を取得する辞書
	27	+ COLOR_MAP = {
	28	+ "normal": COLOR_NORMAL,
	29	+ "fake": COLOR_FAKE,
	30	+ "ice": COLOR_ICE,
	31	+ "trampoline": COLOR_TRAMPOLINE,
	32	+ "trap": COLOR_TRAP
	33	+ }
19	34	
20	35	# =====
21	36	# 1. データ構造 (クラス) の定義

36	51		self._max_charge = 20.0
37	52		self._direction = 1
38	53		
39		-	self._is_clear = False # ゴールしたかどうかのフラグ
	54	+	self._is_clear = False
	55	+	self._on_ice = False
	56	+	self._is_reset = False
40	57		
41	58		# --- ゲッター ---
42	59		def get_image(self): return self._image
49	66		def get_max_charge(self): return self._max_charge
50	67		def get_direction(self): return self._direction
51	68		def get_is_clear(self): return self._is_clear
	69	+	def get_on_ice(self): return self._on_ice
	70	+	def get_is_reset(self): return self._is_reset
52	71		
53	72		# --- セッター ---
54	73		def set_vel_x(self, value): self._vel_x = value
58	77		def set_charge_power(self, value): self._charge_power = value
59	78		def set_direction(self, value): self._direction = value
60	79		def set_is_clear(self, value): self._is_clear = value
	80	+	def set_on_ice(self, value): self._on_ice = value
	81	+	def set_is_reset(self, value): self._is_reset = value
61	82		
62	83		
63	84		class Platform:
64		-	# 足場ごとに色を変えられるように引数追加
65		-	def __init__(self, x, y, w, h, color):
	85	+	def __init__(self, x, y, w, h, color, plat_type="normal"):
66	86		self._image = pygame.Surface((w, h))
67	87		self._image.fill(color)
68	88		self._rect = self._image.get_rect(topleft=(x, y))
	89	+	self._type = plat_type
69	90		
70	91		def get_image(self): return self._image
71	92		def get_rect(self): return self._rect
	93	+	def get_type(self): return self._type
72	94		
73	95		
74	96		# =====
75	97		# 2. クラスを操作する関数
76	98		# =====
77	99		
78	100		def update_player(player, keys, platforms, goal_block):
79		-	"""プレイヤーの物理挙動と状態を更新する関数"""
80		-	
81		-	# ゴール済みの場合は入力を受け付けない（重力で落ちるだけ）
82	101		if not player.get_is_clear():
83		-	# 1. 地面にいる時の処理
84	102		if player.get_on_ground():
85		-	player.set_vel_x(0)
	103	+	
	104	+	if player.get_on_ice():
	105	+	player.set_vel_x(player.get_vel_x() * 0.96)
	106	+	else:
	107	+	player.set_vel_x(0)
86	108		
87	109		if keys[pygame.K_LEFT] or keys[pygame.K_a]:
88	110		player.set_direction(-1)
105	127		player.set_is_charging(False)
106	128		player.set_charge_power(0)
107	129		player.set_on_ground(False)
	130	+	player.set_on_ice(False)
108	131		
109		-	# 2. 重力の適用
110	132		if not player.get_on_ground():
111	133		player.set_vel_y(player.get_vel_y() + GRAVITY)

112	134	
113	135	rect = player.get_rect()
114	136	
115		- # 座標の更新 (X軸) と壁の跳ね返り
116	137	rect.x += player.get_vel_x()
117	138	if rect.left < 0:
118	139	rect.left = 0
123	144	player.set_vel_x(player.get_vel_x() * -0.5)
124	145	player.set_direction(-1)
125	146	
126		- # 座標の更新 (Y軸)
127	147	rect.y += int(player.get_vel_y())
128	148	
129		- # --- 3. ゴール (天井) との当たり判定 ---
130	149	goal_rect = goal_block.get_rect()
131	150	if rect.colliderect(goal_rect):
132		- # 上昇中に天井の底にぶつかったらゴール
133	151	if player.get_vel_y() < 0:
134	152	rect.top = goal_rect.bottom
135	153	player.set_vel_y(0)
136	154	player.set_is_clear(True)
137	155	
138		- # --- 4. 足場との当たり判定 ---
139	156	player.set_on_ground(False)
	157	+ player.set_on_ice(False)
	158	+
140	159	if player.get_vel_y() > 0:
141	160	for plat in platforms:
	161	+ if plat.get_type() == "fake":
	162	+ continue
	163	+
142	164	plat_rect = plat.get_rect()
143	165	if rect.colliderect(plat_rect):
144	166	if rect.bottom <= plat_rect.top + player.get_vel_y() + 1:
	167	+
	168	+
	169	+ if plat.get_type() == "trampoline":
	170	+ rect.bottom = plat_rect.top
	171	+ player.set_vel_y(-25)
	172	+ player.set_on_ground(False)
	173	+ break
	174	+
	175	+ if plat.get_type() == "trap":
	176	+ rect.center = (WIDTH // 2, HEIGHT - 100)
	177	+ player.set_vel_y(0)
	178	+ player.set_vel_x(0)
	179	+ player.set_is_reset(True)
	180	+ break
	181	+
145	181	rect.bottom = plat_rect.top
146	182	player.set_vel_y(0)
147		- player.set_vel_x(0)
148	183	player.set_on_ground(True)
	184	+
	185	+
	186	+ if plat.get_type() == "ice":
	187	+ player.set_on_ice(True)
	188	+ else:
	189	+ player.set_vel_x(0)
	190	break
150	190	
151	191	
156	196	def main():
157	197	pygame.init()
158	198	screen = pygame.display.set_mode((WIDTH, HEIGHT))
159		- pygame.display.set_caption("Jump King - 10 Floors to Goal")
	199	+ pygame.display.set_caption("Jump King - Colored Gimmicks")
160	200	clock = pygame.time.Clock()
161	201	font = pygame.font.SysFont(None, 36)

```

162 | 202 | large_font = pygame.font.SysFont(None, 72)
163 | 203 |
164 | 204 | player = Player()
165 |   | -
166 |   | - # --- 10層分のマップ生成 ---
167 | 205 | platforms = []
168 |   | - platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, GREEN)) # スタート床
169 | 206 | +
170 | 207 | + platforms.append(Platform(0, HEIGHT - 40, WIDTH, 40, COLOR_NORMAL, "normal"))
171 | 208 |
172 | 209 | TOTAL_FLOORS = 10
173 |   | -
174 |   | - # ゴール（天井）のY座標：スタート床から 10層分(800px * 10) 上へ
175 | 210 | goal_y = (HEIGHT - 40) - (HEIGHT * TOTAL_FLOORS)
176 |   | -
177 |   | - # 金色の天井ブロックを生成
178 |   | - goal_block = Platform(0, goal_y, WIDTH, 40, GOLD)
179 |   | -
180 |   | - # ゴールに到達するまで足場を生成し続ける
181 |   | - platform_y = HEIGHT - 150
182 |   | - while platform_y > goal_y + 100:
183 |   | -     w = random.randint(60, 130)
184 |   | -     x = random.randint(0, WIDTH - w)
185 |   | -     platforms.append(Platform(x, platform_y, w, 20, GREEN))
186 |   | -     gap_y = random.randint(80, 130)
187 |   | -     platform_y -= gap_y
188 |   | + goal_block = Platform(0, goal_y, WIDTH, 40, GOLD, "goal")
189 |   | +
190 |   | + for f in range(TOTAL_FLOORS):
191 |   | +     floor_bottom = HEIGHT - (f * HEIGHT)
192 |   | +     floor_top = HEIGHT - ((f + 1) * HEIGHT)
193 |   | +
194 |   | +     floor_plats = []
195 |   | +     plat_y = floor_bottom - 150
196 |   | +
197 |   | +     while plat_y > floor_top + 50:
198 |   | +         w = random.randint(60, 130)
199 |   | +         x = random.randint(0, WIDTH - w)
200 |   | +         floor_plats.append(Platform(x, plat_y, w, 20, COLOR_NORMAL, "normal"))
201 |   | +         gap_y = random.randint(80, 130)
202 |   | +         plat_y -= gap_y
203 |   | +
204 |   | +     if len(floor_plats) >= 2:
205 |   | +         special_type = random.choice(["fake", "ice", "trampoline"])
206 |   | +
207 |   | +         # 10階の場合：ランダム特殊床1つ + トラップ床1つ
208 |   | +         if f == TOTAL_FLOORS - 1:
209 |   | +             specials = random.sample(range(len(floor_plats)), 2)
210 |   | +
211 |   | +             p_spec = floor_plats[specials[0]]
212 |   | +             # 種類に応じた色 (COLOR_MAP[special_type]) を適用
213 |   | +             floor_plats[specials[0]] = Platform(p_spec.get_rect().x, p_spec.get_rect().y,
214 |   | +             p_spec.get_rect().width, 20, COLOR_MAP[special_type], special_type)
215 |   | +
216 |   | +             p_trap = floor_plats[specials[1]]
217 |   | +             # 赤色を適用
218 |   | +             floor_plats[specials[1]] = Platform(p_trap.get_rect().x, p_trap.get_rect().y,
219 |   | +             p_trap.get_rect().width, 20, COLOR_TRAP, "trap")
220 |   | +
221 |   | +         # 1~9階の場合：ランダム特殊床1つ
222 |   | +         else:
223 |   | +             special_idx = random.choice(range(len(floor_plats)))
224 |   | +             p_spec = floor_plats[special_idx]
225 |   | +             floor_plats[special_idx] = Platform(p_spec.get_rect().x, p_spec.get_rect().y,
226 |   | +             p_spec.get_rect().width, 20, COLOR_MAP[special_type], special_type)
227 |   | +
228 |   | +         platforms.extend(floor_plats)
229 |   | +
230 |   | +
231 |   | +
232 |   | +
233 |   | +
234 |   | +
235 |   | +
236 |   | +
237 |   | +
238 |   | +
239 |   | +
240 |   | +
241 |   | +
242 |   | +
243 |   | +
244 |   | +
245 |   | +
246 |   | +
247 |   | +
248 |   | +
249 |   | +

```

186	250	camera_y = 0
187	251	max_height = 0
194	258	
195	259	keys = pygame.key.get_pressed()
196	260	
197	-	# 関数呼び出し (ゴールブロックも渡して判定させる)
198	261	update_player(player, keys, platforms, goal_block)
199	262	
200	263	p_rect = player.get_rect()
201	264	
202	-	# --- カメラ処理 ---
	265	+ if player.get_is_reset():
	266	+ camera_y = 0
	267	+ player.set_is_reset(False)
	268	+
203	269	SCROLL_TOP_MARGIN = 200
204	270	if p_rect.y - camera_y < SCROLL_TOP_MARGIN:
205	271	camera_y = p_rect.y - SCROLL_TOP_MARGIN
213	279	if camera_y > 0:
214	280	camera_y = 0
215	281	
216	-	# --- スコア・階層計算 ---
217	282	current_height = (HEIGHT - 40) - p_rect.bottom
218	283	if current_height > max_height:
219	284	max_height = current_height
220	285	
221	-	# 今いる階層 (800px を 1階 とする)
222	286	current_floor = min((current_height // HEIGHT) + 1, TOTAL_FLOORS)
	287	+ display_floor = max(1, current_floor)
223	288	
224	-	# --- 描画処理 ---
225	289	screen.fill(BLACK)
226	290	
227	-	# ゴールの描画
228	291	goal_rect = goal_block.get_rect()
229	292	goal_draw_y = goal_rect.y - camera_y
230	293	if -100 < goal_draw_y < HEIGHT + 100:
231	294	screen.blit(goal_block.get_image(), (goal_rect.x, goal_draw_y))
232	295	
233	-	# 足場の描画
234	296	for plat in platforms:
235	297	plat_rect = plat.get_rect()
236	298	draw_y = plat_rect.y - camera_y
237	299	if -50 < draw_y < HEIGHT + 50:
238	300	screen.blit(plat.get_image(), (plat_rect.x, draw_y))
239	301	
240	-	# プレイヤーの描画
241	302	player_draw_y = p_rect.y - camera_y
242	303	screen.blit(player.get_image(), (p_rect.x, player_draw_y))
243	304	
244	-	# チャージゲージと方向指示器
245	305	if player.get_is_charging() and not player.get_is_clear():
246	306	gauge_width = (player.get_charge_power() / player.get_max_charge()) * 40
247	-	pygame.draw.rect(screen, RED, (p_rect.centerx - 20, player_draw_y - 15, gauge_width, 8))
	307	+ pygame.draw.rect(screen, COLOR_RED_UI, (p_rect.centerx - 20, player_draw_y - 15, gauge_width, 8))
248	308	
249	309	arrow_x = p_rect.right + 5 if player.get_direction() == 1 else p_rect.left - 10
250	310	pygame.draw.rect(screen, WHITE, (arrow_x, player_draw_y + 15, 5, 5))
251	311	
252	-	# 情報表示 (UI)
253	-	ui_text = font.render(f"Floor: {current_floor} / {TOTAL_FLOORS} Max: {max_height // 10}m", True, WHITE)
	312	+ ui_text = font.render(f"Floor: {display_floor} / {TOTAL_FLOORS} Max: {max_height // 10}m", True, WHITE)
254	313	screen.blit(ui_text, (10, 10))
255	314	

256		-	# ゴールした時の演出
257	315		if player.get_is_clear():
258	316		clear_text = large_font.render("CLEAR!!", True, GOLD)
259	317		text_rect = clear_text.get_rect(center=(WIDTH // 2, HEIGHT // 2 - 50))